

AWSKRUG DEVOPS

DevOps의 다음 고객은 AI입니다

AI 시대, DevOps는 무엇을 제공해야 할까

LG U+ 김보경

PAYMENT ERROR



오전 2:13^{zzz}

결제 실패율 급증

ON-CALL

FIRST LOOK

지금은 조사가 먼저

사람이 화면을 하나씩 열어 보기 전에

최근 배포

오류 로그

런북

AI에게 먼저 조사를 맡기자

REQUEST

원인만 조사

근거 링크 · 운영 변경 금지



오류 로그를 읽어도 될까요?



배포 기록도 읽어도 될까요?



요청은 분명했다

PERMISSION 3/3

런북도 읽어도 될까요?

또 승인해야 했습니다

결국 온콜 엔지니어도 계속 깨어 있었습니다 🙄



PREREQUISITES

프롬프트보다 먼저

에이전트는 우리 시스템을 모른다

도구

서비스 컨텍스트

권한

플랫폼이 먼저 준비한다

PLATFORM RULES

읽을 레포와 차단할 리소스

도구와 컨텍스트만 연결해선 부족하다

읽어도 되는 레포

건드리면 안 되는 리소스

허용 범위를 먼저 정한다

ALREADY HERE

실행 수단은 이미 있다

버튼을 누르는 일은 어렵지 않다

Deploy

Search logs

GET /releases

Deploy · Search · GET

WHY AI?

무엇부터 볼까?

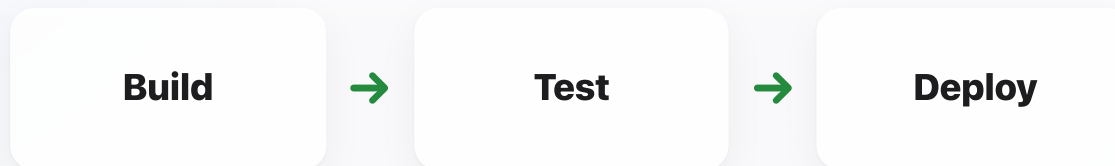
누르는 일과 고르는 일은 다르다

다음에 할 일을 고른다

FIXED STEPS

Commit에서 Deploy까지

Build → Test → Deploy



빠르고 · 비용이 적고 · 결과가 같다

AI RECEIPT

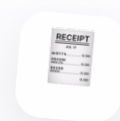
실행은 그대로

요청을 종합적으로 분석한 결과...

설명 4문단

배포는 그대로

다음 단계가 달라질 때만 AI



INCIDENT FLOW

결제 오류

증상마다 먼저 볼 곳이 달라진다

오류 로그

최근 배포

DB 쿼리

외부 API

정해진 순서 없음

NEXT QUESTION

결과 → 다음 질문

방금 확인한 사실이 다음 조사를 바꾼다

다음에 볼 것은 조사하면서 정해진다

ROLE

모든 분기를 코드로?

가능하지만 유지보수하는 사람이 먼저 지친다

모든 분기를 미리 작성

상황에 맞춰 다음 조사를 선택

AI는 다음 조사를 고른다

SMALL WORKFLOW

매번 손으로 처리한다

사소하지만 자주 흐름을 끊는 일

규칙 찾기



브랜치 만들기



검증 정리

자동화 후보

BEFORE AI

선행 비용 > 절약 시간

인터뷰, 합의, 예외 목록, 유지보수

몇 명만 쓰는 일이라 포기

AFTER AI

포기했던 흐름을 다시 본다

사람마다 순서가 조금 달라도 처리할 수 있다

작은 일도 자동화할 만해진다

TRADE-OFF

코드와 AI는 비용 구조가 다르다

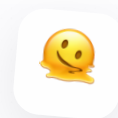
빠름, 비용, 재현성을 따로 비교해야 한다

코드 · 빠르고 결과가 같다

AI · 느리고 탐색 순서가 달라진다

용도가 다르다

SIDE QUEST



“

원인 조사 + 옆 파일 TODO

친절은 고맙지만 지금은 오전 2:13

”

예상 밖의 작업부터 늘어난다

NOT A CONTEST

경쟁시키지 말고 역할을 나눈다

유연함이 필요한 구간에만 AI를 둔다

실행 전체를 맡기지 않는다

EXPLORE

답이 없으면 다른 자료로

목표는 고정하고 탐색 순서는 바꾼다

로그

문서

변경 이력

모르면 사람에게 질문

EXECUTE

탐색과 실행을 분리

탐색 순서가 달라도 완료 조건은 같다

AI · 유연한 탐색

코드 · 결정론적 실행

확정된 실행 = 코드·정책

DEVOPS

유연한 탐색 + 결정론적 실행

둘 사이의 허용 범위를 플랫폼에 담는다

허용 범위를 설계하는 DevOps

DEVOPS CUSTOMERS

+1

DevOps의 고객이 한 명 늘었다

개발자

운영자

AI

AI 에이전트

THESIS

DevOps의 다음 고객은 AI입니다

서비스 고객이 아니라 플랫폼을 직접 쓰는 사용자

이 사용자가 일할 환경을 만들어야 한다

BOUNDARY

자율성 ≠ 권한 확대

권한을 많이 준다고 더 자율적인 건 아니다

첫 원칙은 읽기 전용

LEAST PRIVILEGE



읽을 수 있지만 바꿀 수 없다

운영 변경 API는 아예 제공하지 않는다

✓ 로그 · 허용

✓ 배포 기록 · 허용

✓ 런북 · 허용

✗ 운영 변경 · 차단

권한은 좁고 분명하게

SCOPED AUTONOMY

읽기 전용 장애 조사만

근거 수집과 원인 후보 정리부터 맡긴다

권한 범위를 먼저 정한다

INVESTIGATION RESULT

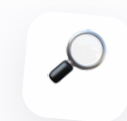
조사 근거를 한곳에 모은다

배포 시각과 로그 패턴을 함께 본다

02:07 · payment-api v184 근거 ↗

02:11 · timeout 8.4배 근거 ↗

원문으로 바로 이동



EVIDENCE FIRST

확신보다 원문 링크

원인 후보마다 확인한 근거를 붙인다

후보 A · 외부 결제사 지연

후보 B · 연결 설정 변경

확신 92%보다 원문 링크 1개

UNKNOWN

아직 모름

확인하지 못한 항목을 숨기지 않는다

모른다는 사실도 조사 결과다

AGENT SAYS

“

지금 롤백할 수 있습니다

근거를 찾은 뒤 제안까지는 할 수 있다

”

하지만 실행 권한은 별개다

SEPARATION

좋은 제안 ≠ 실행 권한

그럴듯한 답변이 실행 권한을 주지는 않는다

AI · 원인과 조치 제안

정책 · 허용 여부 결정

제안과 권한을 분리한다

APPROVAL

AI → 사람 → 자동화

사람이 근거를 확인한 뒤 승인한다

조사



검토·승인



기존 자동화

실행 권한은 계속 분리해 둔다

APPROVED

승인 완료

검증된 롤백 자동화를 실행한다

운영 절차는 그대로

EXECUTION GUARD

agent shell: disabled

에이전트가 운영 셸을 직접 열지 않는다

target · payment-api / v183

on-failure · stop + page

정해진 인터페이스만 연다

GUARDRAILS

설명 가능 · 복구 가능

실패하면 멈추고 사람을 호출한다

✓ 대상 고정

✓ 오류율 기준

✓ 실패 시 중단

✓ 되돌릴 수 있음

실행은 결정론적으로

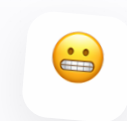
COMMAND RESULT

200 OK

에이전트: 끝났습니다

명령은 성공했다

CUSTOMER RESULT



18.7%

결제 실패율



결제만 빼고 끝났습니다

DONE MEANS DONE

고객이 다시 결제하면 끝

명령 성공보다 고객 문제가 풀렸는지 본다

✓ 테스트

✓ 오류율

✓ 응답 시간

✓ 결제 성공률

완료 조건을 플랫폼에 넣는다

NEW SESSION



이전 판단을 이어받지 못한다

왜 그렇게 결정했는지 알지 못한다

이전 세션 · 운영 변경 금지

새 세션 · 어떤 서비스인가요?

이전 컨텍스트 없음

LONG TASK

요약될수록 조건이 빠진다

처음 준 조건과 작은 예외가 빠질 수 있다

저장소 구조



서비스 사정



v184 예외



컨텍스트가 조금씩 빠진다

PROMPT LIMIT

새 세션마다 팀 소개부터?

프롬프트만 길게 쓰는 방식은 오래가기 어렵다

컨텍스트는 환경에 둔다

CONTEXT PLATFORM

설명 대신 구조

에이전트가 스스로 찾아 읽을 자료를 만든다

AGENTS.md

Service catalog

Runbook

Work log

컨텍스트도 미리 준비한다

JUST IN TIME

지금 필요한 컨텍스트만

모든 문서를 한꺼번에 넣지 않는다

현재 작업

관련 서비스

남은 위험

필요한 순간에 찾아 읽는다

PRODUCTIZE CONTEXT

규칙과 결정은 남겨 둔다

레포와 서비스 카탈로그에서 찾아 읽게 한다

프롬프트보다 구조

FIRST TIME

처음 보는 예외

AI와 사람이 함께 조사한다

관찰



가설



확인

가설을 넓히고 근거를 확인한다

SECOND TIME

같은 원인·같은 조치

다음에는 같은 추론을 반복하지 않는다

테스트



정책



도구

AI가 찾아낸 반복은 자동화한다

MUSINSA TRACK



AX인재전쟁

MUSINSA

무슨 문제를 풀까?

해결책을 만들기 전에 문제를 찾았다

열린 과제의 첫 질문



3 HOURS

사람이 다 읽기엔 너무 넓었다

AI로 더 많은 자료를 빠르게 훑었다

뉴스룸·보도

상품·리뷰

채용 공고

법령

자료는 넓게 보고 결론은 미뤘다

FROM HYPOTHESIS TO TOOL

AI 요약은 결론이 아니다

원문과 대조한 뒤 규칙 기반 플러그인으로 만들었다

AI 조사



원문 확인



규칙 기반 도구

무신사 트랙 1등

EVIDENCE

AI가 한 말은 증거가 아니다

그럴듯함과 확신은 근거가 아니다

가설로만 다룬다

CROSS-CHECK

한 출처에서 멈추지 않는다

틀렸을 때 비용이 클수록 원문을 읽는다

공식 자료



다른 보도



사용자 반응

근거를 여러 출처에서 확인한다

DECISION

AI는 후보를 넓히고 사람은 고른다

최종 실행은 규칙 기반 도구에 맡긴다

AI · 가설을 넓힌다

사람 · 문제를 고른다

판단 책임은 사람이 진다

EXPERIMENT

작업과 검토를 분리

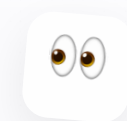
별도 세션에서 요구사항, 변경, 테스트를 다시 읽는다

작업 에이전트

Spotter

초반의 오해를 다시 확인한다

SAME BLIND SPOT



“

문제없습니다·저도요

세션을 둘로 나뉘도 같은 오해에 합의할 수 있다

”

별도 ≠ 독립

MEASURE IT

안전장치가 아니라 실험

효과가 있다고 가정하지 않는다

같은 판단으로 틀린 비율 · 오탐 · 누락 · 시간 · 비용

START SMALL



온콜 장애 조사부터 시작한다

처음에는 조사까지만 맡긴다

운영 변경 권한 없음

OUTPUT SCHEMA

출력 스키마와 측정 기준을 정한다

첫 근거가 도착한 시간과 사람이 수정한 분량을 기록한다

원인 후보 · 근거 근거 ↗

반대 근거 · 모르는 것 근거 ↗

근거와 반대 근거를 함께 남긴다

CONTROLLED GROWTH

비교한 뒤 한 단계씩 넓힌다

AI를 쓰지 않은 비슷한 사건과 비교한다

AI 없이 처리한 사건



성과 확인



범위 확장

성과가 확인될 때만 범위를 넓힌다

NEXT INCIDENT



오전 2:13

미리 묶어 둔 읽기 권한으로 조사를 시작한다

권한 요청 0 · 운영 변경 0

CALLBACK

DevOps의 고객이 한 명 늘었다

DevOps의 역할은 그대로다

DevOps의 다음 고객은 AI입니다



LinkedIn · 김보경

[linkedin.com/in/
bogyeong-kim-366a17246](https://www.linkedin.com/in/bogyeong-kim-366a17246)